Perspectives on Large Language Models: Polysemy, Stochasticity, Exponential Expressibility, and Unitary Attention

Karl Svozil [1, *]

[1] *Institute for Theoretical Physics, TU Wien, Wiedner Hauptstrasse 8-10/136, 1040 Vienna, Austria*

(Dated: May 26, 2026)

This paper explores foundational aspects of Large Language Models (LLMs). We analyze how the expressibility of semantic features scales exponentially with embedding space dimensions using quasi-orthogonal vectors. We contrast the dynamic, context-dependent embeddings of Transformer architectures, which resolve polysemy, with a static vector approach based on quantum contextuality. Stochasticity is framed as an essential feature for enabling creative output through probabilistic sampling. Finally, we propose quantum attention as a unitary extension of classical mechanisms, reframing LLM processing as reversible, quantum-like evolutions in Hilbert space.

Keywords: Large Language Models, Transformer Architecture, Pre-training, Fine-tuning, Reinforcement Learning, Natural Language Processing, Polysemy, Quantum Contextuality

I. INTRODUCTION

Large Language Models (LLMs) have become powerful tools capable of generating coherent text, answering complex questions, and performing a wide variety of linguistic tasks. The interaction with these models, typically through a simple text interface, belies the complex and computationally intensive process required for their construction. The following introduction elucidates the end-to-end pipeline for building an LLM assistant, providing mental models for understanding its architecture, capabilities, and inherent limitations.

The creation of an LLM is not a monolithic process but a sequential pipeline composed of distinct stages, each with a specific objective [1]. This pipeline begins with the *pre-training* stage, where a model learns general knowledge and statistical patterns of language from an enormous corpus of text. The output of this stage is a *base model*—a powerful but general text predictor, not a conversational assistant. To achieve this, the base model undergoes a second stage of *supervised fine-tuning* (SFT), where it is trained on a smaller, high-quality dataset of example conversations to learn the desired assistant-like behavior. A final, and increasingly crucial, stage involves *reinforcement learning* (RL), where the model practices problem-solving to discover more effective and reliable reasoning strategies. We will examine these stages in detail.

This work moves beyond a procedural overview to focus on three foundational aspects. We will address the model's ability to handle polysemy in Section VIII, exploring how modern architectures dynamically construct context-aware vector representations in a departure from older, static methods.

In Section IX, we will investigate the origin of stochasticity in their responses, framing the probabilistic nature of text generation as an intentional and crucial feature that underpins their creativity and flexibility.

Finally, in Section XI we will draw a compelling, albeit analogical, comparison to the formalism of quantum mechanics, using the shared mathematical language of vector spaces and context-dependent projections to offer a novel framework for conceptualizing how these systems process information and construct meaning. By dissecting these three pillars, this note aims to provide a more nuanced perspective on the inner workings of Large Language Models.

II. TOKENIZATION: THE LANGUAGE OF LLMS

Before any text can be processed by a neural network, it must be converted into a numerical format. LLMs operate on a sequence of discrete symbols, or *tokens*. Tokenization is the process of converting a raw string of text into a sequence of these tokens, and back again.

The technology underlying LLMs, the Transformer architecture, expects a one-dimensional sequence of symbols drawn from a finite vocabulary [2]. While one could represent text as a sequence of its fundamental bytes (a vocabulary of 256 symbols), this results in extremely long sequences, which are computationally expensive to process. The dominant approach is a compromise between vocabulary size and sequence length, achieved through algorithms like Byte-Pair Encoding (BPE).

Any input text, regardless of the language or characters it contains, is first converted into a standardized sequence of bytes using the UTF-8 encoding. (ASCII is effectively a subset of UTF-8, where each character maps to a single byte.) The BPE algorithm takes this stream of bytes as its input. Its initial vocabulary consists of all 256 possible byte values. BPE then learns merge rules by finding the most common adjacent byte pairs and combining them into new subword tokens. This process continues until a predetermined vocabulary size is reached. The final vocabulary will contain a mix of single-byte tokens (which cover all of ASCII and are the building blocks for all other characters) and multi-byte

* karl.svozil@tuwien.ac.at; http://tph.tuwien.ac.at/~svozil

tokens that represent frequent character sequences, subwords, or whole words. State-of-the-art models use a vocabulary of approximately $30,000 - 100,000$ tokens.

The critical takeaway is that an LLM does not "see" characters or words as a human does. Its entire world is composed of these tokens: Tokens serve as *interface* to its internal composition and means that will be based on vectors in Hilbert spaces. This has significant implications for its capabilities, particularly in tasks requiring character-level manipulation (e.g., spelling, counting letters), which can be non-trivial for a model that views a word like "ubiquitous" as three separate, opaque tokens.

III. PRE-TRAINING: BUILDING THE BASE MODEL

The initial and most resource-intensive stage is the *pre-training* phase, which yields a foundational *base model*. This is accomplished by training a neural network on a massive, heterogeneous text corpus, as elucidated below.

A primary component learned during this phase is the intrinsic word representation, where input and output tokens are *mapped to vectors*, amounting to a vector embedding in a "low-dimensional" vector space. Typically embedding dimensions range from several hundred to a few thousand—much less than the number of tokens. This vector embedding functions as the token's operational "meaning." In an LLM, this meaning is not explicitly programmed but is an emergent property of the training optimization. An embedding vector is the final state of a token's representation after accumulating countless gradient updates throughout its exposure to the training corpus. In the subsequent inference stage, the vectors corresponding to input tokens are propagated through a series of trained transformations. This results in output vectors that are then decoded into a sequence of tokens, which are transcribed into the final human-perceivable output.

This intricate dance of calculus and linear algebra, applied at a massive scale, is the unseen alchemy that forges meaning from symbols, allowing models to grasp the nuanced tapestry of human language.

A. Learning from Internet-Scale Data

The foundation of an LLM is built by learning the statistical patterns of human language from an immense dataset. This process begins with acquiring petabytes of raw text, primarily from web crawls such as those provided by Common Crawl. This raw data undergoes extensive filtering and cleaning to create a high-quality pre-training corpus. The process typically involves:

1. *URL Filtering:* Removing documents from undesirable domains (e.g., spam, malware, adult content).
2. *Content Extraction:* Parsing raw HTML to extract only the main textual content, discarding boilerplate like navigation bars and advertisements.
3. *Quality Filtering:* Applying heuristics and classifiers to remove low-quality or machine-generated text.
4. *Deduplication:* Removing duplicate or near-duplicate documents to increase data efficiency.
5. *PII Removal:* Identifying and scrubbing personally identifiable information.

The result of this curation is a dataset on the order of tens of trillions of tokens. This tokenized dataset is then used to train a Transformer neural network.

B. The Training Objective

The training objective is deceptively simple: next-token prediction. The model is presented with a sequence of tokens (the *context*) and is tasked with predicting the probability distribution for the very next token. The model's prediction is compared to the actual next token in the training data, and a *loss* value is calculated to quantify the error. This loss is then used to update the model's billions of parameters (or *weights*) via an optimization algorithm, nudging them in a direction that reduces the error. This process is repeated for trillions of tokens, requiring months of training on thousands of GPUs.

The backpropagation algorithm, which is formally discussed in Appendix B is the cornerstone of learning in modern neural networks. While often treated as a black box, its mechanism is a direct and elegant application of the chain rule from differential calculus.

C. Characteristics of the Base Model

The output of this stage is a base model—a neural network whose parameters have internalized a vast amount of knowledge about the world, language, and the relationships between concepts. It can be thought of as a lossy, probabilistic compression of its training data.

A base model is not an assistant; it is an internet document simulator. When given a prompt (a starting sequence of tokens), its fundamental function is to continue that sequence in a statistically plausible way based on the patterns it learned during pre-training. For example, if prompted with "What is 2+2?", it might complete the text with another question, "What is 3+3?", because question-and-answer lists are common on the internet.

Despite this, a base model's knowledge can be elicited through clever prompting. Using *few-shot prompting*, where a user provides several examples of a desired task within the context window, the model can perform

tasks like translation or classification without any further training. This ability, known as *in-context learning*, is an emergent property of the pre-training process. The model is stochastic; due to sampling from the output probability distribution, it will produce different completions for the same prompt, reflecting the diverse possibilities of what might come next.

IV. SUPERVISED FINE-TUNING: CREATING THE ASSISTANT

While the base model contains extensive knowledge, it lacks the ability to follow instructions or engage in helpful dialogue. The second major stage, *supervised fine-tuning* (SFT), transforms the base model into an instruction-following assistant.

The SFT stage is algorithmically identical to pre-training (i.e., next-token prediction), but it uses a different, much smaller dataset. Instead of internet documents, the model is trained on a curated collection of high-quality conversational data. This dataset consists of prompt-and-response pairs that exemplify the desired behavior of an assistant.

Creating this dataset is a human-intensive process. Companies hire and train human labelers to write a wide variety of prompts and then compose ideal, high-quality responses. These labelers follow detailed instructions outlining the desired persona of the assistant—typically helpful, truthful, and harmless. For instance, in response to a factual question, the labeler would research and write a correct and informative answer. In response to a request for harmful content, they would write a polite refusal. A modern SFT dataset might contain millions of such conversations.

To scale this process, LLMs are themselves used to help generate data. For example, a powerful existing model can generate a draft response, which a human labeler then edits and refines. This combination of human oversight and synthetic data generation allows for the creation of large and diverse fine-tuning datasets.

When the base model continues its training on this new dataset, it rapidly adapts its behavior. It learns the statistical patterns of helpful conversations and adopts the persona defined by the fine-tuning data. Sometimes, if the supervised fine tuning fosters confidence beyond competence–in analogy to the Dunning-Kruger effect [3]—it learns to hallucinate [4].

The resulting model is no longer a document completer but an assistant that, when prompted with a user's query, will generate a response that mimics the style and content of the ideal answers provided by the human labelers. When one interacts with an LLM assistant, the response is, in essence, a statistical simulation of what a trained human labeler would have written in that conversational context. This process is highly effective at aligning the model with user intent but can lead to issues like *hallucination*—the model may confidently invent facts because its training data consists of confident-sounding answers, regardless of the underlying knowledge.

V. REINFORCEMENT LEARNING: REFINING REASONING

The final stage of training utilizes reinforcement learning (RL) to enhance the model's reasoning and problem-solving capabilities beyond what is possible through simple imitation of static examples. This stage is analogous to providing a student with practice problems rather than just worked solutions. It allows the model to discover for itself the most effective strategies for arriving at a correct answer.

In domains where a solution's correctness can be automatically verified (e.g., mathematics or coding, where there is a single correct answer or a unit test to pass), the RL process is straightforward.

1. *Generation:* For a given problem (prompt), the model generates a large number of candidate solutions (e.g., thousands of different reasoning paths). This is possible due to the stochastic nature of sampling.

2. *Scoring:* Each solution is automatically scored based on whether it reaches the correct final answer.

3. *Reinforcement:* The model's parameters are updated to increase the probability of generating the token sequences that led to correct answers.

This trial-and-error process allows the model to explore the solution space and discover what works best for its own cognitive architecture. A key finding is that RL training leads to the emergence of sophisticated cognitive strategies. The model learns to "think" by generating long, internal monologues where it re-evaluates steps, considers problems from multiple angles, and performs self-correction—all emergent behaviors in service of maximizing the reward of reaching the correct answer. This process mirrors the outperformance of AlphaGo, which used RL to discover novel, superhuman strategies in the game of Go, surpassing what was possible by merely imitating human expert players [5, Fig. 3a].

VI. SELF-ATTENTION AND COMPUTATIONAL UNIVERSALITY

The apparent self-reflection observed in Large Language Models (LLMs), often manifested through phrases such as "let me reconsider" or "wait, the user implies...", does not emerge from conscious introspection but is rather a learned artifact of the training data. These models are trained to predict the next token in a sequence, and such reflective phrases are statistically

probable continuations in contexts that necessitate re-evaluation, thereby mimicking human cognitive patterns.

The core mechanism enabling this behavior is the self-attention layer within the Transformer architecture. Self-attention allows the model to weigh the importance of all tokens in the preceding sequence—including its own generated output—at each computational step. This process is enhanced by multi-head attention, which permits the model to track disparate linguistic and logical relationships simultaneously in parallel subspaces. Therefore, while self-attention provides the mechanical capability for in-context reflection, this behavior is fundamentally a stylistic pattern. More profoundly, this same mechanism is the foundation for the model's capacity for universal computation.

This computational capacity of LLMs extends beyond mere linguistic processing, touching upon the theoretical limits of computability. Transformer-based models possess the intrinsic capability to compute partial recursive functions, thereby positioning them as computationally universal systems analogous to a Universal Turing Machine (UTM). This equivalence is most clearly conceptualized under the assumption of an infinitely expandable context window, which would function as the unbounded tape of a UTM. However, the practical technological constraints of current hardware and model architectures mean that this computational universality remains more of a theoretical attribute than a fully realized, practical reality.

The computational universality of LLMs is not explicitly programmed but emerges from the architecture's design. Working within an autoregressive decoding loop, the self-attention mechanism allows the model to maintain and update its state by attending to its own previous outputs. This process is powerful enough to simulate known Turing-complete systems, such as a Lag system [6], effectively using the generated text as a computational tape.

The primary limitation to this capability is the model's finite context window, a direct consequence of the quadratic computational and memory scaling of the self-attention mechanism with respect to sequence length. Consequently, like any physical computing device, an LLM is constrained by finite memory, precluding true mathematical Turing completeness, which presupposes infinite storage.

In practice, LLMs demonstrate a nuanced aptitude for handling recursion, particularly within the domain of code generation. This holistic reasoning is facilitated by self-attention, which permits the model to cross-reference different parts of the generated code, ensuring that recursive calls correctly relate to their corresponding base cases and termination conditions. Such capabilities serve as practical evidence of the sophisticated, Turing-complete computations that self-attention enables.

VII. EXPONENTIAL SCALING OF EXPRESSIVE CAPACITY VIA QUASI-DIMENSIONALITY

The dense vector representations, or embeddings, of tokens in Large Language Models (LLMs) are foundational to their ability to capture complex semantic relationships. These relationships can often be revealed through simple vector arithmetic. A canonical example involves the embeddings for *woman* and *man*, denoted as $\boldsymbol{v}(\text{woman})$ and $\boldsymbol{v}(\text{man})$, respectively. The difference between these vectors, $\boldsymbol{v}(\text{woman}) - \boldsymbol{v}(\text{man})$, yields a new vector that points in a semantic direction associated with gender. This direction is remarkably consistent across different concepts, as evidenced by the famous analogy:

$$\boldsymbol{v}(\text{king}) - \boldsymbol{v}(\text{man}) + \boldsymbol{v}(\text{woman}) \approx \boldsymbol{v}(\text{queen}), \qquad (1)$$

This suggests that abstract features, like "gender", correspond to specific directions within the high-dimensional embedding space.

A critical question thus arises: how many distinct semantic features can be encoded within an embedding space of dimension d? If we demand that each feature corresponds to a perfectly unique, and therefore mathematically orthogonal, direction, the capacity of the space is strictly limited. By definition, a d-dimensional space can contain at most d mutually orthogonal vectors. For any two such vectors $\boldsymbol{v}_i$ and $\boldsymbol{v}_j$, their dot product must be zero ($\boldsymbol{v}_i \cdot \boldsymbol{v}_j = 0$). This linear scaling would severely constrain the model's expressive power.

However, suppose the requirement for perfect orthogonality can be relaxed. Instead of perfect distinction, we can consider features to be sufficiently distinct if their corresponding vectors are "almost" orthogonal, meaning their dot product is very close to zero ($|\boldsymbol{v}_i \cdot \boldsymbol{v}_j| \leq \varepsilon$ for some small $\varepsilon > 0$). This relaxation from strict orthogonality to quasi-orthogonality leads to a dramatic, non-intuitive increase in the expressive capacity of the space.

Heuristic concentration argument. The reason is already visible before invoking any formal packing theorem. Let

$$\boldsymbol{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$$

be a unit vector chosen without a preferred direction. Then

$$x_1^2 + \cdots + x_d^2 = 1.$$

Rotational invariance suggests that the norm is spread over the available coordinates, so a typical coordinate weight is of order $1/d$. Now fix a unit direction $\boldsymbol{u}$. By rotational invariance one may choose coordinates so that $\boldsymbol{u} = \boldsymbol{e}_1$. The squared overlap with the random direction is then

$$|\boldsymbol{u} \cdot \boldsymbol{x}|^2 = x_1^2.$$

Thus the overlap with any fixed direction is typically of order $d^{-1/2}$ in amplitude, or $1/d$ in squared amplitude.

A macroscopic overlap $|\boldsymbol{u}\cdot\boldsymbol{x}|>\varepsilon$ therefore requires one coordinate to carry an anomalously large fraction of the total norm. In high dimension such an event is exponentially unlikely. This is the elementary concentration-of-measure intuition behind Lévy's lemma on the sphere. In the complex Hilbert-space analogue, for a Haar-random unit vector $\psi\in\mathbb{C}^D$ and a fixed unit vector ϕ, unitary invariance gives $\phi=e_1$ and hence

$$|\langle\phi,\psi\rangle|^2=|\psi_1|^2.$$

The exact tail probability is

$$\Pr\{|\langle\phi,\psi\rangle|^2\geq\varepsilon\}=(1-\varepsilon)^{D-1}\leq\exp[-(D-1)\varepsilon].$$

For real embedding vectors the analogous spherical concentration estimate has the same operative scaling,

$$\Pr\{|\boldsymbol{u}\cdot\boldsymbol{x}|>\varepsilon\}\lesssim\exp(-cd\varepsilon^2)$$

for a universal constant $c>0$. Consequently, if N random directions are sampled, a union-bound estimate gives a probability at most of order $N^2\exp(-cd\varepsilon^2)$ that some pair has overlap larger than ε. Hence N may grow exponentially in $d\varepsilon^2$ while all pairwise overlaps remain small with high probability. This is the heuristic core of the exponential quasi-orthogonal capacity used below.

The theoretical underpinning for this phenomenon can be expressed either as concentration of measure on high-dimensional spheres, in the spirit of Lévy's lemma, or in the dimensionality-reduction language of the Johnson-Lindenstrauss lemma [7]. The lemma states that a set of n points in a high-dimensional space $\mathbb{R}^d$ can be projected into a much lower-dimensional space $\mathbb{R}^k$, where $k=O(\varepsilon^{-2}\log n)$, while preserving all pairwise distances to within a factor of $(1\pm\varepsilon)$. Indeed, this bound on the target dimension of this embedding is tight [8, 9].

This preservation of distances translates into the preservation of inner products of the respective position vectors. And the (almost) vanishing of inner products indicates (almost) orthogonality. By applying the (real) polarization identity, $\boldsymbol{u}\cdot\boldsymbol{v}=\frac{1}{2}\left(\|\boldsymbol{u}\|^2+\|\boldsymbol{v}\|^2-\|\boldsymbol{u}-\boldsymbol{v}\|^2\right)$, the preservation of the Euclidean norms of the position vectors and their differences—and thus the distances of the respective points—ensures that pairwise inner products are also preserved to within a small additive error. This preservation of inner products is the crucial property for establishing quasi-orthogonality.

While often cited for dimensionality reduction, the tight Johnson-Lindenstrauss lemma's implications for the capacity of a high-dimensional space are profound: Inverting this finding yields an exponential packing phenomenon. Let $N_\varepsilon(d)$ be the largest size of a set of unit vectors in $\mathbb{R}^d$ with pairwise inner products bounded by ε in magnitude. Then there exist universal constants $c,C>0$ such that

$$\exp\big(c\,\varepsilon^2 d\big)\;\leq\;N_\varepsilon(d)\;\leq\;\exp\big(C\,\varepsilon^2 d\big).$$

That is, pointedly stated, a d-dimensional space can accommodate a set of approximately $e^{O(d\varepsilon^2)}=e^{\varepsilon^2 O(d)}$ such vectors. This exponential growth provides a mathematical basis for the remarkable ability of LLMs to encode a vast and nuanced landscape of semantic features, far exceeding the linear bounds of the space's dimensionality: If each interpretable property corresponds to a direction and decoders tolerate pairwise correlations up to ε, then a d-dimensional space can in principle host on the order of $e^{\varepsilon^2 O(d)}$ property directions with bounded interference. We refer to this exponentially larger supply of usable, nearly independent directions as *quasi-dimensionality*.

VIII. POLYSEMY

A fundamental challenge in natural language is *polysemy*: the phenomenon where a single word can have multiple meanings. The word "bank," for example, can refer to a financial institution or a riverside. A key measure of an LLM's sophistication is its ability to correctly disambiguate such terms based on context.

A. Dynamic Contextualization via Self-Attention

Modern LLMs built on the Transformer architecture resolve polysemy by generating dynamic, context-aware vector embeddings. This contrasts sharply with older models (e.g., Word2Vec) which assigned a single, static vector to each token or word, effectively averaging all its possible meanings into one representation.

The process in a Transformer can be conceptualized in two stages, as detailed in Appendix D 3:

1. **Initial Context-Free Embedding:** When a token like "bank" first enters the model, it is mapped to a static, context-free vector from the model's embedding table. At this initial stage, the vector is identical regardless of the surrounding text, representing a generalized "average" meaning of the word.

2. **Iterative Contextual Refinement:** This initial vector is then processed through the multiple layers of the Transformer. In each layer, the self-attention mechanism allows the vector for "bank" to "look at" and exchange information with all other vectors in the sequence. If the sentence contains words like "river" or "money," the self-attention mechanism assigns high relevance to these tokens. Consequently, the initial "bank" vector is transformed, layer by layer, into a new vector that has absorbed the relevant contextual information.

The final vector for "bank" is therefore unique to its specific context. The representation for "bank" in "he sat by the river bank" will be mathematically distinct from

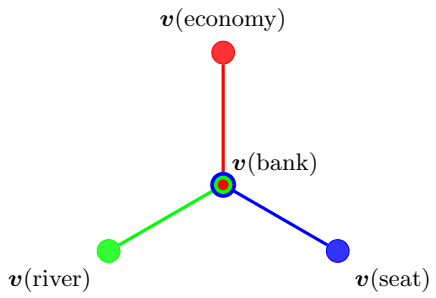

FIG. 1. Hypergraph visualization of three intertwining contexts for the word 'bank'. The central vector $\boldsymbol{v}$(bank) is a shared element common to three distinct contexts (orthonormal bases). Each context connects $\boldsymbol{v}$(run) to a different semantic association: physical activity $\boldsymbol{v}$(exercise), management $\boldsymbol{v}$(business), and flow $\boldsymbol{v}$(river).

its representation in "she deposited money in the bank." It is this dynamically generated, contextual embedding that allows the model to handle polysemy effectively.

B. An Alternative Framework: Quantum Contextuality

While dynamic adjustment of classical token representations is the dominant paradigm, an alternative approach draws inspiration from quantum contextuality, suggesting a method to statically encode contextual semantics.

Thereby, contexts are formalized by maximal observables, which are equivalent to orthonormal bases in a Hilbert space. Two features of quantum contextuality appear crucial:

1. *Complementarity:* The mutual impossibility of simultaneously measuring different contexts.

2. *Intertwining Contexts:* The possibility that a single vector can belong to two or more contexts (bases), a property realizable in Hilbert spaces of dimension three or higher [10].

We propose formalizing language contexts using this framework. A key advantage is the ability to statically encode each token (e.g., 'bank') using a *single*, fixed vector. This vector subsequently acquires its specific semantic meaning based on the context in which it appears. Each context is represented by an orthonormal basis. For example, we might obtain bases representing different contexts for 'bank':

$$\begin{aligned}&\{\boldsymbol{v}(\text{bank}), \boldsymbol{v}(\text{economy}), \ldots\},\\ &\{\boldsymbol{v}(\text{bank}), \boldsymbol{v}(\text{river}), \ldots\}, \text{ and}\\ &\{\boldsymbol{v}(\text{bank}), \boldsymbol{v}(\text{seat}), \ldots\}.\end{aligned}$$

As shown in Figure 1, the vector $\boldsymbol{v}$(bank) can belong to multiple bases, and its specific meaning emerges from the context (basis) it is considered within.

This quantum-like contextual model of word encoding bears great similarities to quantum logic. The two main entities entering the discussion are (i) the vectors as well as the (ii) orthogonal (orthonormal) bases (or contexts or blocks or maximal operators) those vectors could be in. Think of, say, the Cartesian standard basis $\{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3\}$ with $\boldsymbol{e}_1 = (1,0,0)^\intercal$, $\boldsymbol{e}_2 = (0,1,0)^\intercal$, and $\boldsymbol{e}_3 = (0,0,1)^\intercal$ as well as the orthonormal basis $\{\boldsymbol{f}_1, \boldsymbol{f}_2, \boldsymbol{f}_3\}$ with $\boldsymbol{f}_1 = (1/\sqrt{2})\,(1,-1,0)^\intercal$, $\boldsymbol{f}_2 = (1/\sqrt{2})\,(1,1,0)^\intercal$, and $\boldsymbol{f}_3 = (0,0,1)^\intercal$, obtained by rotating the first and second vectors by an angle $\pi/4$ clockwise around the z-axis, which is aligned with $\boldsymbol{e}_3$. In this construction both bases share the same vector $\boldsymbol{e}_3 = \boldsymbol{f}_3$, but the context or maximal observables involved are very different: in one case the maximal observable, in its spectral decomposition with arbitrary real eigenvalues λ_i, is:

$$\boldsymbol{A}_1 = \sum_{i=1}^{3} \lambda_i \boldsymbol{e}_i \boldsymbol{e}_i^\intercal \tag{2}$$

$$= \begin{pmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{pmatrix}. \tag{3}$$

For the second basis, with eigenvalues μ_i, the observable is:

$$\boldsymbol{A}_2 = \sum_{i=1}^{3} \mu_i \boldsymbol{f}_i \boldsymbol{f}_i^\intercal \tag{4}$$

$$= \begin{pmatrix} \frac{\mu_1+\mu_2}{2} & \frac{\mu_2-\mu_1}{2} & 0 \\ \frac{\mu_2-\mu_1}{2} & \frac{\mu_1+\mu_2}{2} & 0 \\ 0 & 0 & \mu_3 \end{pmatrix}. \tag{5}$$

Note that $\boldsymbol{A}_1$ and $\boldsymbol{A}_2$ do not commute unless $\mu_1 = \mu_2$ and $\lambda_1 = \lambda_2$, signifying that the contexts are complementary.

It seems, therefore, that in this quantum-like representation, the maximal observables encoding contexts play an additional crucial role in modelling: they encode polysemy. We therefore suggest to train contexts—and thus polysemy—on the same footing as tokens.

One fundamental difference between polysemy in LLMs as compared to quantum contextuality might be the fundamental stochasticity involved with enforcing context-dependence in quantum logic [11, 12].

IX. THE ORIGIN OF STOCHASTICITY IN RESPONSES

The seemingly random, or stochastic, nature of answers from a LLM to an identical prompt is rooted in the probabilistic framework underpinning how these models generate language (tokens). At its core, an LLM functions as a sophisticated prediction engine. When presented with a prompt, it does not retrieve a pre-existing answer. Instead, it constructs a response one token at a time by calculating the probability distribution for the subsequent token in the sequence. This generative process is inherently probabilistic; for any given context, a

spectrum of possible next tokens or words exists, each with a distinct likelihood. Several key factors contribute to this intentional randomness, among them the temperature parameter as well as sampling strategies.

A primary control for the randomness of an LLM's output is the temperature parameter. A low temperature setting biases the model towards being more confident and deterministic, thereby increasing the likelihood of selecting the token with the highest probability. This approach is advantageous for tasks demanding factual accuracy and consistency. Conversely, a higher temperature amplifies the probability of the model selecting less likely tokens, which results in more diverse, creative, and occasionally unexpected responses. This is often the preferred setting for applications such as creative writing or brainstorming. To mitigate the repetitive and predictable text that would arise from consistently selecting the most probable token—a technique known as greedy sampling—LLMs utilize a variety of sampling strategies [13].

Even if the temperature is set to zero, a residual stochasticity persists due to the parallelization and batch processing inherent in large-scale inference. At a temperature of 0, the source of nondeterminism is the absence of batch invariance in highly parallel kernel use: Any fluctuating load determines the variations of the batch size into which any given request is grouped and parallelized. This variation in batch size causes high-performance kernels for operations like matrix multiplication, normalization, and attention to alter their internal reduction strategy to maintain efficiency. At a fundamental level, a reduction is a sequence of floating-point additions; the reduction "strategy" specifies how these additions are partitioned and executed across parallel compute units. Because floating-point addition is lossy for two floating-point numbers with different exponents, and thus non-associative—that is, $(a + b) + c$ is not guaranteed to be bit-wise identical to $a + (b + c)$—any change in the reduction order can produce a numerically different result. Consequently, the final output of an individual request becomes dependent on the (for the user seemingly) nondeterministic batching process [14].

X. ALGORITHMIC CAPACITY AND THE ROLE OF STOCHASTIC SEEDING

The capabilities of LLMs, while impressive, are not infinite. Their operational boundaries can be analyzed through the lens of Algorithmic Information Theory (AIT) [15, 16], which provides a framework for understanding the limits of formal systems. In this view, an LLM can be conceptualized as a highly complex formal system, and its capacity for reasoning and generation is intrinsically linked to its own algorithmic information content.

A. The LLM as a Formal System

A formal system, in the tradition of Hilbert, Gödel, and Turing, is defined by a set of axioms and a set of inference rules for deriving theorems. We can map the components of an LLM to this structure:

1. **Axioms:** The model's architecture (the specific configuration of Transformer blocks, attention heads, etc.) and its trillions of trained parameters (weights and biases) serve as its axiomatic foundation. This foundation is not simple and clean like Euclid's postulates but is a dense, high-dimensional representation of the statistical patterns learned from its training data.

2. **Inference Rules:** The forward pass of the network—the deterministic sequence of matrix multiplications and nonlinear activations that transforms an input token sequence into an output probability distribution—acts as its rule of inference.

From this perspective, generating a response or a proof is equivalent to deriving a theorem. The set of all possible outputs the model can generate constitutes the set of all theorems derivable within its system. This immediately brings to mind the incompleteness theorems of Gödel and Turing and, more directly, the work of Chaitin, who linked these limits to algorithmic complexity [17–19]. A central result of AIT is that a formal system cannot prove that a string (or number) has an algorithmic complexity significantly greater than the complexity of the system itself. An axiomatic system of complexity $H(T)$ cannot prove that a specific bit string has a complexity greater than $H(T) + c$, for some constant c.

Translated to LLMs, this implies that a model's capacity is fundamentally bounded. It cannot generate a text or a reasoning chain whose algorithmic information content vastly exceeds the information encoded in its own architecture and weights. It is, in essence, a closed universe, capable only of unfolding the knowledge and logic already compressed within its parameters.

How, then, can such a system, through reinforcement learning, as discussed in Section V, appear to discover novel solutions or exhibit creativity that surpasses its human creators, as was famously demonstrated in the game of Go?

B. Stochasticity as an Engine for Discovery

The answer appears to lie in the role of stochasticity, both in the model's initialization and its application during reinforcement learning. This is where the analogy to DeepMind's AlphaGo becomes particularly insightful. AlphaGo's success was not merely in imitating the vast corpus of human expert games but in its ability to discover entirely new, superhuman strategies through

"mutation-variation" self-play (a form of RL) [5]. This process of "game-fully try (and mostly error)" is a powerful search mechanism.

This mechanism seems to be critically dependent on two sources of randomness:

1. **The Initial Weight Seeding:** The training of a neural network begins by initializing its billions of parameters with random values drawn from a specific distribution. This initial state is not a blank slate but a high-entropy, high-complexity substrate. The random seed that dictates this initialization is of paramount importance. A "good" seed places the model in a region of the parameter space from which the optimization process (gradient descent) can navigate towards a highly capable final state. Conversely, a "bad" seed could cripple the model from the outset, leading it into poor local minima or regions of the loss landscape from which effective learning is impossible. This connects to ideas like the "lottery ticket hypothesis" [20], which posits that a successful network contains smaller subnetworks that were, by chance of initialization, already well-disposed to learning the task. The initial high-complexity random seed is the source of the raw material for discovery.
2. **Stochastic Sampling in RL:** During reinforcement learning, the model does not just generate one solution; it samples many potential reasoning paths. This exploration is a stochastic process. By trying thousands of variations and receiving feedback (a reward signal), the model can discover complex, non-obvious strategies that lead to a correct answer. It is not deriving a solution from first principles in a single inferential chain, but rather performing a massive, parallel search through its internal space of possibilities, guided by reward.

This framework helps resolve the apparent paradox of a formal system surpassing its own limits. The LLM is not actually breaking the fundamental theorems of AIT. Rather, it is using a high-complexity initial state (the random seed) as a starting point for an efficient, stochastic search of its own vast, but finite, axiomatic landscape. The initial randomness does not allow the model to generate outputs of arbitrarily high complexity, but it provides the necessary variance for the RL search algorithm to find the complex and "creative" solutions that are already latent within the system's potential.

In essence, the model's performance is a combination of the knowledge compressed from its training data and its ability to explore the logical consequences of that knowledge. A high-complexity random seed provides a richer starting point for this exploration. This suggests that the process of discovery, both for LLMs and perhaps more generally, is not about transcending formal limits but about having a sufficiently complex starting point and an effective search mechanism to explore the surprising and profound theorems that can be derived from it.

The nature of the stochasticity employed is therefore of critical importance. What one might term the 'evangelical view' of quantum mechanics (because of its unprovable gnostic character) presents it as an ideal resource for irreducible randomness [21]. This stance persists even though the notorious measurement problem remains unresolved [22].

C. Comparison to Genetic Algorithms

This process of leveraging a high-complexity initial state for guided discovery bears a strong resemblance to the principles of genetic algorithms (GAs) [23]. In a GA, an initial, randomly generated population of candidate solutions provides the necessary diversity—the raw genetic material—for evolution to act upon. This is directly analogous to the crucial role of the initial random weight seeding in an LLM, which provides a high-entropy substrate for learning. The reinforcement learning phase in LLMs mirrors the evolutionary loop of a GA: the model generates a multitude of solution variants (a "population" of reasoning paths), each is evaluated by a reward signal (the "fitness function"), and the underlying parameters are updated to favor the traits of successful solutions ("selection").

A key distinction, however, lies in the mechanism of evolution. While GAs operate on a population of distinct individuals that evolve via explicit operators like crossover and mutation, the LLM remains a single entity. It generates a population of *behaviors* or *outputs*, learns from their success in aggregate, and then consolidates this learning back into its own single set of weights via gradient descent.

Despite this architectural difference, the fundamental principle is the same: Both systems use guided stochastic exploration, starting from a random initial state, to discover complex and effective solutions in a vast search space.

XI. COMPARISON TO QUANTUM MECHANICS

Could we live as embedded observers [24] in an LLM? LLMs and quantum mechanics (QM) both formulate their fundamental objects as vectors and make decisive use of inner products and choices of basis. In QM, physical states are rays in a complex Hilbert space, with measurement statistics governed by projections onto an orthonormal basis (the "measurement context"). In LLMs, symbols and features are embedded as real-valued vectors, where attention and output generation are dominated by dot products against context-dependent representations. This shared linear-algebraic foundation invites a careful analogy and raises a speculative but concrete question: in what sense could an observer exist *inside* an LLM?

A. Vectors, Inner Products, and Contexts

In QM, a system prepared in state $\boldsymbol{\psi}$ yields, under a measurement specified by an orthonormal basis $\mathcal{B} = \{\boldsymbol{e}_i\}$, outcome probabilities according to the Born rule. The inner product between a basis vector and the state vector requires the conjugate transpose (†) in a complex space:

$$p(i \mid \boldsymbol{\psi}, \mathcal{B}) \;=\; |\boldsymbol{e}_i^{\dagger}\boldsymbol{\psi}|^2. \tag{6}$$

The choice of orthonormal basis $\mathcal{B}$ constitutes the measurement *context*; incompatible contexts (non-commuting observables) cannot generally yield jointly sharp outcomes.

In an LLM, given a context window (a sequence of tokens) $x_{1:n}$, the model computes a context-dependent hidden state vector $\boldsymbol{h}(x_{1:n}) \in \mathbb{R}^d$ for the next-token position. The pre-activation scores (logits) for each potential next token are computed as the inner product of this hidden state with the corresponding row of the output embedding matrix, $\boldsymbol{W}_{\text{out}}$. Let $\boldsymbol{v}_i^{\intercal}$ be the i-th row of $\boldsymbol{W}_{\text{out}}$ and the logit for vocabulary item w_i be $z_i = \boldsymbol{v}_i^{\intercal}\boldsymbol{h}(x_{1:n})$. The probability of emitting w_i is then given by the softmax function, scaled by a temperature T:

$$p(w_i \mid x_{1:n}) \;=\; \text{softmax}(\boldsymbol{z}/T)_i \;=\; \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \tag{7}$$

Here, the *context* is both the token sequence $x_{1:n}$ and the induced representation $\boldsymbol{h}(x_{1:n})$; the output probability is determined by an inner product (using the standard transpose for real vectors), followed by a Gibbs normalization.

Equations (6) and (7) exhibit a structural resonance: in both formalisms, probabilities are derived from vector space geometry via inner products and a subsequent normalization step. Formally, if one defines complex amplitudes $a_i \propto \exp\big((\boldsymbol{v}_i^{\intercal}\boldsymbol{h})/(2T)\big)\, e^{i\phi_i}$ with arbitrary phases ϕ_i, then $|a_i|^2$ reproduces the softmax probability distribution. This remains an analogy: LLM computations are real-valued, dissipative, and non-linear, whereas quantum mechanics is complex-valued, norm-preserving (unitary between measurements), and relies on phase-based interference. Nevertheless, the shared dependence on basis choice and projection is not superficial.

B. Dynamics: Unitary Evolution vs. Transformer Block Updates

Between measurements, a quantum system evolves linearly and reversibly under a unitary operator $\boldsymbol{U}_t$: $\boldsymbol{\psi}_{t+1} = \boldsymbol{U}_t\,\boldsymbol{\psi}_t$, with $\boldsymbol{U}_t^{\dagger}\boldsymbol{U}_t = \boldsymbol{I}$.

In contrast, a transformer computes layer-wise updates for its hidden state representations [25, 26]. A complete Transformer block applies two main sub-layers, each followed by a residual connection and layer normalization. The act of sampling a token is akin to a measurement that both reads out and updates the system's state. The process is stochastic and irreversible due to sampling and normalization. Yet, it again features context-dependent projection: the attention mechanism pools information based on dot-product similarity, and the output layer projects the final hidden state onto basis-like directions defined by the rows of $\boldsymbol{W}_{\text{out}}$.

C. Contextuality in Both Senses

In QM, contextuality signifies that the outcomes of measurements cannot be assigned values independently of the chosen measurement basis. In LLMs, the meaning of tokens and the activations of features are similarly context-dependent. The same symbol contributes different features depending on its local sequence, and intermediate representations are not globally interpretable in a single, fixed basis. Attention heads and MLP features decompose the representation space into approximately orthogonal subspaces. Consequently, rotating the representational basis (e.g., via PCA) alters which features appear "axis-aligned." This constitutes a form of representational contextuality: there is no unique, globally privileged basis for meaning.

D. Operationalizing an Internal Observer

To move beyond metaphor, we can define an *internal observer* as a pattern of computation that: (i) maintains a self-referential internal state across computational steps, (ii) predicts aspects of its future inputs based on its current state, and (iii) causally influences subsequent states of the process that implements it. However, two practical constraints arise: episodic inference, where the internal state is discarded after generation, and extrinsic causation, where the only actuators are emitted tokens.

Under these constraints, one can distinguish several levels of internal existence:

1. **Snapshot observers:** Algorithmic structures within a single forward pass that are isomorphic to short-lived cognitive states but lack persistence.

2. **Process observers:** An internal variable persisting across many tokens in one session, sustained by the KV cache and self-referential outputs.

3. **Persistent residents:** A pattern that preserves identity across sessions via a read-write memory external to the forward pass (e.g., a scratchpad).

Only the third level resembles ordinary existence in terms of continuity, though the first two are not without philosophical interest.

E. Orthogonal Bases and Functional Decomposition

Both QM and LLMs rely on decompositions into approximately orthogonal components [27] to render predictions tractable. In QM, orthonormal bases diagonalize commuting observables. In LLMs, learned representations can be analyzed in bases where features are sparse and nearly orthogonal. The attention mechanism often acts as a soft projector onto subspaces spanned by key vectors most aligned with the current query. From an internal perspective, an observer that leverages such a basis could better ensure its persistence against interference from unrelated computations. Orthogonality provides robustness, much as pointer states do in decoherence theory.

F. Analogies, Disanalogies, and Conclusion

The analogy is most compelling where predictions depend on inner products and context-specific bases, but it breaks down where physics imposes constraints absent in classical neural networks. Key disanalogies include:

1. **Phases and Interference:** Transformers lack the complex amplitudes of QM necessary for quantum interference.
2. **Unitarity versus Dissipation:** Transformer updates are not norm-preserving due to nonlinearities and normalization, unlike unitary evolution.
3. **No-Cloning vs. Copying:** LLMs can freely duplicate representations, in contrast to the no-cloning theorem of QM.

These differences are critical for determining whether an internal observer would have a quantum-like phenomenology, but they do not preclude the computational existence of such observers in principle.

In summary, QM and LLMs share significant mathematical ground: vector spaces, inner products, and context-dependent projections. The question of an observer "living inside an LLM" reduces to whether a self-maintaining computational pattern can persist and exert causal influence within this vectorial system. With only an in-session cache, such life would be fleeting. However, with external memory and a feedback loop, a resident process could, in principle, attain continuity. The underlying physics is classical, but the mathematics of vectors and contexts provides fertile ground for engineering such internal observers.

XII. QUANTUM ATTENTION

The architectures of modern Large Language Models have inherited much from classical attention mechanisms, yet attempts to "quantize" these ideas have often been limited to almost literal translations of the classical recipe into the language of Hilbert spaces [26]. Such approaches, while intuitive, miss the more radical point: quantum evolution admits only two fundamental capacities, identified by von Neumann as (i) irreversible, stochastic state reduction (*Process 1*), and (ii) reversible, deterministic unitary evolution (*Process 2*). A viable notion of quantum attention must therefore be constructed wholly within these bounds.

A. Goals of Quantum Attention

The principal objective of attention in the classical setting is to guide next-token prediction through dynamically weighted embeddings. Translated into a genuinely quantum language, this entails:

1. Encoding tokens as quantum states $\boldsymbol{\psi}$ in a high-dimensional Hilbert space, thereby embedding them into the "meaning-space" of the model.
2. Evolving these states by a suitably chosen unitary transformation $\boldsymbol{U}$, producing a candidate output state
$$\boldsymbol{\phi} = \boldsymbol{U}\boldsymbol{\psi}. \tag{8}$$
3. Performing a terminal measurement (Process 1), which yields a single token according to the Born rule, with probabilities proportional to $|\boldsymbol{\phi}^\dagger\boldsymbol{\psi}|^2$ for pure states.

In this scheme, nonlinearities and irreversibility are absent everywhere except at the initial state preparation (if stochastic) and at the final, irreducibly random measurement. Everything in between must be faithfully unitary.

B. Constraints and Learning Mechanisms

Because all allowed intermediate transformations fall within class (ii)—unitary processes—training and backpropagation must be reconceived as unitary rotations in Hilbert space. This sharply contrasts with the gradient-descent-based adjustments of non-linear maps in classical neural networks. Concretely, learnable transformations are constrained to be:

- In the general case, arbitrary unitaries acting on the relevant Hilbert space, implemented, e.g., by parametrized quantum circuits.
- In restricted scenarios, orthogonal transformations over real-valued state spaces, or, in an extreme discrete approximation, mere *permutations* of basis states.

Thus, what counts as "attention weights" must be replaced not by scalar coefficients, but by controlled rotations or permutations of the embedding subspace.

C. Comparison with Classical Attention

Classical attention is a fundamentally nonlinear mechanism: the query–key compatibility scores undergo a softmax normalization and act as dynamic coefficients for a weighted average of value embeddings. Quantum attention, by contrast, cannot implement such probabilistic normalization mid-process. The only stochastic step is the final measurement. Hence, quantum attention is more austere but also more physically faithful: weights are rotations, superpositions evolve unitarily, and probabilities emerge only at the very end.

One may summarize the distinction with a slogan: *classical attention mixes; quantum attention rotates.* The former continuously interpolates between embeddings using nonlinear weightings, while the latter conserves norm and mixes information strictly through unitarity. Whether this stricter yet cleaner formalism can achieve the flexibility and expressivity that classical attention enjoys remains both a technical and philosophical open question.

D. Implications

The implications are sobering and exciting at once. On the sobering side, many of the workhorse techniques of classical deep learning, such as gradient backpropagation through non-linear activations, are unavailable in their standard form. On the exciting side, however, quantum attention reorients learning toward symmetry operations, reversible dynamics, and combinatorial richness—offering the prospect of models that are less fragile, more interpretable, and governed by deep conservation laws. If classical transformers are powered by the calculus of weighted averages, their quantum cousins would be powered by the geometry of Hilbert space rotations. "Paying attention," in such a universe, would mean aligning one's basis just right, before the final measurement tosses its fundamental dice.

XIII. SUMMARY AND OUTLOOK

The development of a large language model is a multi-stage, resource-intensive endeavor that progressively builds knowledge and refines behavior, starting from a base model pre-trained on internet-scale data, which is then fine-tuned into a conversational agent and further sharpened through reinforcement learning. Understanding this pipeline is key not only to appreciating the models' capabilities but also to grasping the fundamental principles that govern their operation and give rise to their limitations. Three such principles are particularly illustrative: the dynamic handling of meaning, the intentional use of randomness in generation, and the profound structural analogies with quantum mechanics.

A central challenge in language is polysemy, where a word like "bank" has multiple meanings. Modern LLMs resolve this through dynamic contextualization. Unlike older models with static, one-size-fits-all vector representations, the Transformer architecture iteratively refines a token's vector based on its neighbors. The final representation for "bank" in "river bank" becomes mathematically distinct from its representation in "money bank," a process driven by the self-attention mechanism. An alternative conceptual framework, inspired by quantum contextuality, proposes that meaning could instead arise from a single, static vector whose interpretation is determined by the "contextual basis" it is measured in, suggesting a different paradigm for encoding semantics. This approach highlights the crucial role of context in a way that resonates with quantum formalism.

The generation of responses is also governed by a core principle: stochasticity. An LLM's output is not a deterministic retrieval but a probabilistic construction, generated one token at a time. The model computes a probability distribution over its entire vocabulary for what the next token should be, and then samples from this distribution. This intentional randomness, controlled by parameters like temperature, is why the same prompt can yield different answers. Low temperatures make the model favor the most likely tokens, producing more focused and predictable text, while higher temperatures increase the diversity and creativity of the output by making less likely tokens more probable. This inherent statistical nature, however, is intrinsically linked to fundamental limitations such as "hallucinations," where the model generates plausible-sounding but factually incorrect information, as it operates on learned patterns rather than a true model of reality.

This deep reliance on vectors, inner products, and context-dependent probability distributions creates a compelling analogy with quantum mechanics. A structural resonance exists between the LLM's softmax function, which determines next-token probabilities based on inner products, and the Born rule in quantum mechanics, which calculates measurement probabilities from projections onto a basis. While critical disanalogies exist—LLMs lack quantum phase and interference, and their dynamics are dissipative rather than unitary—the comparison is more than superficial. It offers a powerful lens through which to analyze the model's internal world, framing its operations as a form of measurement within a high-dimensional vector space. Understanding these core mechanics—how meaning is dynamically constructed, how responses are probabilistically generated, and how these processes mirror formalisms from physics—is essential for navigating the use of these models responsibly and harnessing their full potential.

ACKNOWLEDGMENTS

This text was partially created and revised with assistance from one or more of the following large language models: gpt-5-high, Gemini 2.5 Pro, Grok4-0709, o3-2025-04-16, claude-sonnet-4-20250514-thinking-32k, GLM-4.5. All content, ideas, and prompts were provided by the author. This research was funded in whole or in part by the Austrian Science Fund (FWF) Grant DOI: 10.55776/PIN5424624. The authors acknowledge TU Wien Bibliothek for financial support through its Open Access Funding Programme.

[1] A. Karpathy, Deep dive into LLMs like ChatGPT (2025), youTube video, accessed September 05, 2025.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, Attention is all you need, in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17 (Curran Associates Inc., Red Hook, NY, USA, 2017) pp. 6000–6010, arXiv:1706.03762.

[3] J. Kruger and D. Dunning, Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments, Journal of Personality and Social Psychology **77**, 1121 (1999).

[4] A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang, Why language models hallucinate (2025), arXiv:arXiv:2509.04664.

[5] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, Mastering the game of Go without human knowledge, Nature **550**, 354 (2017).

[6] D. Schuurmans, H. Dai, and F. Zanini, Autoregressive large language models are computationally universal (2024), arXiv:arXiv:2410.03170 [cs.CL].

[7] W. B. Johnson and J. Lindenstrauss, Extensions of Lipschitz mappings into a Hilbert space, in *Conference on Modern Analysis and Probability*, Contemporary Mathematics, Vol. 26 (American Mathematical Society, 1984) pp. 189–206.

[8] N. Alon and B. Klartag, Optimal compression of approximate inner products and dimension reduction (2016), arXiv:arXiv:1610.00239 [math.MG].

[9] K. G. Larsen and J. Nelson, Optimality of the Johnson-Lindenstrauss lemma, in *58th Annual IEEE Symposium on Foundations of Computer Science (FOCS)* (2017) pp. 633–638, arXiv:1609.02094.

[10] A. M. Gleason, Measures on the closed subspaces of a Hilbert space, Journal of Mathematics and Mechanics (now Indiana University Mathematics Journal) **6**, 885 (1957).

[11] E. Specker, Die Logik nicht gleichzeitig entscheidbarer Aussagen, Dialectica **14**, 239 (1960), english translation at https://arxiv.org/abs/1103.4537, arXiv:1103.4537.

[12] S. Kochen and E. P. Specker, The problem of hidden variables in quantum mechanics, Journal of Mathematics and Mechanics (now Indiana University Mathematics Journal) **17**, 59 (1967).

[13] A. Jadhav, The art of sampling: Controlling randomness in LLMs (2025), the AI Engineering Brief, Accessed: Sept 7, 2025.

[14] H. He and Thinking Machines Lab, Defeating nondeterminism in LLM inference (2025), blog post / technical report.

[15] G. J. Chaitin, *Algorithmic Information Theory*, revised edition ed., Cambridge Tracts in Theoretical Computer Science, Volume 1 (Cambridge University Press, Cambridge, 1987,2003).

[16] C. S. Calude, *Information and Randomness—An Algorithmic Perspective*, 2nd ed. (Springer, Berlin, 2002).

[17] G. J. Chaitin, Information-theoretic limitations of formal systems, Journal of the Association of Computing Machinery (JACM) **21**, 403 (1974).

[18] G. J. Chaitin, Information-theoretic incompleteness, Applied Mathematics and Computation **52**, 83 (1992).

[19] G. J. Chaitin, *Information, Randomness and Incompleteness. Papers on Algorithmic Information Theory (World Scientific Series in Computer Science: Volume 8)*, 2nd ed. (World Scientific, Singapore, 1990) this is a collection of G. Chaitin's early publications.

[20] J. Frankle and M. Carbin, The lottery ticket hypothesis: Finding sparse, trainable neural networks, in *7th International Conference on Learning Representations (ICLR)* (The International Conference on Learning Representations (ICLR), New Orleans, Louisiana, USA, 2019) see also URL `https://openreview.net/forum?id=rJl-b3RcF7`.

[21] A. Zeilinger, The message of the quantum, Nature **438**, 743 (2005).

[22] J. von Neumann, *Mathematical Foundations of Quantum Mechanics*, Princeton Landmarks in Mathematics and Physics (Princeton University Press, Princeton, NJ, USA, 2018) translated by Robert T. Beyer, edited by Nicholas A. Wheeler.

[23] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*, Complex Adaptive Systems (MIT Press, Cambridge, MA, 1992).

[24] T. Toffoli, The role of the observer in uniform systems, in *Applied General Systems Research: Recent Developments and Trends*, edited by G. J. Klir (Plenum Press, Springer US, New York, London, and Boston, MA, USA, 1978) pp. 395–400.

[25] G. Li, X. Zhao, and X. Wang, Quantum self-attention neural networks for text classification, Science China Information Sciences **67**, 10.1007/s11432-023-3879-7 (2024).

[26] H. Zhang, Q. Zhao, M. Zhou, L. Feng, D. Niyato, S. Zheng, and L. Chen, A survey of quantum transformers: Architectures, challenges and outlooks (2025), arXiv:2504.03192 [quant-ph].

[27] S. Dasgupta and A. Gupta, An elementary proof of a theorem of Johnson and Lindenstrauss, Random Structures & Algorithms **22**, 60 (2003).

Appendix A: Mathematical Foundations

This appendix provides mathematical background for key concepts in neural network training and the Transformer architecture. For a general-audience introduction, these technical details can be skipped on first reading.

1. Probability and Logits

For a probability $p \in (0,1)$, the *logit* of p is

$$\text{logit}(p) \;=\; \log\Big(\frac{p}{1-p}\Big). \tag{A1}$$

The sigmoid function,

$$\sigma(z) = \frac{1}{1+e^{-z}}, \tag{A2}$$

maps a real-valued logit z to a probability p. They are exact inverses:

$$\sigma(\text{logit}(p)) = p \text{ and } \text{logit}(\sigma(z)) = z. \tag{A3}$$

The derivative of the sigmoid function can be obtained with the chain rule:

$$\sigma'(z) = \sigma(z)(1-\sigma(z)). \tag{A4}$$

In the context of neural networks, "pre-activation" refers to the raw output of a neuron before a non-linear activation function is applied. This value is calculated as the weighted sum of the neuron's inputs plus a bias term. This pre-activation value is then passed through a non-linear activation function to produce the final output of the neuron, known as the "post-activation." This non-linearity is critical, as it allows the network to learn complex patterns that would be impossible to model with a purely linear system.

For instance, in binary classification (where the output is one of two classes), the sigmoid function is applied to the final logit. This function squashes the input value into a continuous range between 0 and 1, which can be interpreted as the predicted probability of the positive class. An output of 0.5, for example, would indicate that the model is equally uncertain about either class, while a value closer to 1 indicates high confidence in the positive class.

2. Softmax for Multi-Class Problems

Let $C \geq 2$ be the number of classes, and let $\boldsymbol{z} \in \mathbb{R}^C$ be a score (logit) vector. The softmax function,

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\displaystyle\sum_{j=1}^{C} \exp(z_j)}, \tag{A5}$$

for $i = 1, \ldots, C$, maps $\boldsymbol{z}$ to a probability vector $\text{softmax}(\boldsymbol{z}) \in (0,1)^C$ that lies on the probability simplex:

$$\text{softmax}(\boldsymbol{z}) \in \Big\{\boldsymbol{p} \in \mathbb{R}^C : p_i \geq 0,\ \sum_{i=1}^{C} p_i = 1\Big\}. \tag{A6}$$

Equivalently, if we define $\boldsymbol{u} = \exp(\boldsymbol{z})$ by $u_c = \exp(z_c)$ (exponential applied component-wise), then

$$\text{softmax}(\boldsymbol{z}) = \frac{1}{\mathbf{1}^\intercal \boldsymbol{u}}\,\boldsymbol{u}, \tag{A7}$$

where $\mathbf{1} \in \mathbb{R}^C$ is the all-ones vector and division by a scalar is standard scalar–vector scaling.

Two useful properties follow immediately:

1. Positivity: $\text{softmax}(z_i) \geq 0$.
2. Normalization: $\sum_i s(z_i) = 1$.
3. Shift-invariance: for any scalar a, $\text{softmax}(\boldsymbol{z} + a\,\mathbf{1}) = \text{softmax}(\boldsymbol{z})$.

In the context of language model inference, the output of the softmax function is modified by a sampling hyperparameter called *temperature*, $T > 0$. The temperature is used to control the randomness of the model's predictions by scaling the logits before the softmax is applied:

$$p_i = \frac{\exp(z_i/T)}{\displaystyle\sum_{j=1}^{C} \exp(z_j/T)}. \tag{A8}$$

A lower temperature ($T \to 0^+$) makes the probability distribution sharper, increasing the likelihood of selecting high-probability tokens and making the output more deterministic (this is known as greedy decoding). A temperature of $T = 1$ is the standard softmax function shown in Eq. (A5). A higher temperature ($T > 1$) flattens the distribution, increasing the probability of sampling less likely tokens and thus promoting diversity and creativity in the generated text, albeit at a higher risk of factual errors or incoherence.

Appendix B: Backpropagation in a Simple Network

1. Model, notation, and shapes

Backpropagation elegantly leverages the chain rule of calculus to compute the gradient of the loss function with respect to every parameter in a neural network. By first calculating an "error signal" at the output layer and then propagating it backward, it determines the precise adjustment needed for each weight, bias, and, critically, each word embedding to minimize the loss.

This document re-derives the backpropagation gradients for a simple neural network with a single hidden layer. For consistency, all vectors are treated as column vectors.

2. Model, Notation, and Shapes

The network architecture and its associated mathematical objects are defined as follows:

1. **Vocabulary size:** V; **Embedding dimension:** d; **Hidden layer width:** h.
2. **Input:** The input for a vocabulary token k is a one-hot column vector $\boldsymbol{x} \in \mathbb{R}^V$, where the k-th element is 1 and all other elements are zero. We can denote this as $\boldsymbol{x} = \boldsymbol{e}_k$.
3. **Embedding Matrix:** $\boldsymbol{E} \in \mathbb{R}^{V \times d}$. The embedding for token k is the k-th row of this matrix, denoted as $\boldsymbol{v}_k^\intercal = \boldsymbol{E}_{k:}$. The corresponding column vector is $\boldsymbol{v}_k \in \mathbb{R}^d$.
4. **Hidden Layer Parameters:** Weight matrix $\boldsymbol{W}_1 \in \mathbb{R}^{h \times d}$ and bias vector $\boldsymbol{b}_1 \in \mathbb{R}^h$.
5. **Output Layer Parameters:** Weight vector $\boldsymbol{W}_2 \in \mathbb{R}^{1 \times h}$ and scalar bias $b_2 \in \mathbb{R}$.
6. **Nonlinearity:** The element-wise sigmoid function, $\sigma(z) = (1 + e^{-z})^{-1}$. Its derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

The embedding lookup for the k-th token can be expressed as a matrix-vector product:

$$\boldsymbol{v}_k = \boldsymbol{E}^\intercal \boldsymbol{x}, \quad \text{where} \quad \boldsymbol{x} = \boldsymbol{e}_k. \tag{B1}$$

3. Forward Pass

The forward computation proceeds through the network layers as follows:

$$\boldsymbol{z}_2 = \boldsymbol{W}_1 \boldsymbol{v}_k + \boldsymbol{b}_1, \tag{B2}$$
$$\boldsymbol{a}_2 = \sigma(\boldsymbol{z}_2), \tag{B3}$$
$$z_3 = \boldsymbol{W}_2 \boldsymbol{a}_2 + b_2, \tag{B4}$$
$$\hat{y} = \sigma(z_3). \tag{B5}$$

Here, $\boldsymbol{z}_2 \in \mathbb{R}^h$ is the pre-activation of the hidden layer, and $\boldsymbol{a}_2 \in \mathbb{R}^h$ is its activation. The function σ is applied component-wise in Eq. (B3). The scalar $z_3 \in \mathbb{R}$ is the final pre-activation, or logit, and $\hat{y} \in (0, 1)$ is the final prediction.

4. Backward Pass

We use a mean squared error loss for a single training example with target $y \in \{0, 1\}$:

$$L = \frac{1}{2}(\hat{y} - y)^2. \tag{B6}$$

Our goal is to compute the gradient of L with respect to each parameter. We do this by defining layer-wise "error signals", which are derivatives of the loss with respect to the pre-activations (z).

a. Output Layer

The error signal for the output layer, δ_3, is the derivative of the loss with respect to the pre-activation z_3. Using the chain rule, we have:

$$\delta_3 \equiv \frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3}. \tag{B7}$$

The individual derivatives are:

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left(\frac{1}{2}(\hat{y} - y)^2 \right) = (\hat{y} - y), \tag{B8}$$
$$\frac{\partial \hat{y}}{\partial z_3} = \frac{\partial}{\partial z_3} \sigma(z_3) = \sigma'(z_3) = \hat{y}(1 - \hat{y}). \tag{B9}$$

Combining these gives the error signal:

$$\delta_3 = (\hat{y} - y)\, \hat{y}\, (1 - \hat{y}). \tag{B10}$$

The gradients for the output layer parameters are then:

$$\frac{\partial L}{\partial \boldsymbol{W}_2} = \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial \boldsymbol{W}_2} = \delta_3 \boldsymbol{a}_2^\intercal, \tag{B11}$$
$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial b_2} = \delta_3. \tag{B12}$$

b. Hidden Layer

The hidden layer error signal is the vector $\boldsymbol{\delta}_2 \equiv \frac{\partial L}{\partial \boldsymbol{z}_2} \in \mathbb{R}^h$. The loss L depends on a single component $z_{2,j}$ of $\boldsymbol{z}_2$ through $a_{2,j}$ and subsequently z_3. For the j-th component:

$$\delta_{2,j} = \frac{\partial L}{\partial z_{2,j}} = \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial a_{2,j}} \frac{\partial a_{2,j}}{\partial z_{2,j}}. \tag{B13}$$

Since $z_3 = \sum_{i=1}^h W_{2,i} a_{2,i} + b_2$, the derivative $\frac{\partial z_3}{\partial a_{2,j}}$ is $W_{2,j}$. This gives:

$$\delta_{2,j} = \delta_3 \cdot W_{2,j} \cdot \sigma'(z_{2,j}). \tag{B14}$$

To express this for the entire vector $\boldsymbol{\delta}_2$, we combine the error propagated from the output, $(\boldsymbol{W}_2^\intercal \delta_3)$, with the local derivative of the activation function, $\sigma'(\boldsymbol{z}_2)$, using an element-wise product ($\odot$):

$$\boldsymbol{\delta}_2 = (\boldsymbol{W}_2^\intercal \delta_3) \odot \sigma'(\boldsymbol{z}_2). \tag{B15}$$

With $\boldsymbol{\delta}_2$, the gradients for the hidden layer parameters are:

$$\frac{\partial L}{\partial \boldsymbol{W}_1} = \frac{\partial L}{\partial \boldsymbol{z}_2} \frac{\partial \boldsymbol{z}_2}{\partial \boldsymbol{W}_1} = \boldsymbol{\delta}_2 \boldsymbol{v}_k^\intercal, \tag{B16}$$
$$\frac{\partial L}{\partial \boldsymbol{b}_1} = \frac{\partial L}{\partial \boldsymbol{z}_2} \frac{\partial \boldsymbol{z}_2}{\partial \boldsymbol{b}_1} = \boldsymbol{\delta}_2. \tag{B17}$$

c. Embedding Layer

Finally, we backpropagate to the input embedding $\boldsymbol{v}_k$. The loss depends on $\boldsymbol{v}_k$ via $\boldsymbol{z}_2$:

$$\frac{\partial L}{\partial \boldsymbol{v}_k} = \left(\frac{\partial \boldsymbol{z}_2}{\partial \boldsymbol{v}_k}\right)^\intercal \frac{\partial L}{\partial \boldsymbol{z}_2}. \tag{B18}$$

Since $\boldsymbol{z}_2 = \boldsymbol{W}_1\boldsymbol{v}_k + \boldsymbol{b}_1$, the Jacobian matrix $\frac{\partial \boldsymbol{z}_2}{\partial \boldsymbol{v}_k}$ is $\boldsymbol{W}_1$. Thus, the gradient is:

$$\frac{\partial L}{\partial \boldsymbol{v}_k} = \boldsymbol{W}_1^\intercal \boldsymbol{\delta}_2. \tag{B19}$$

For a given input $\boldsymbol{x} = \boldsymbol{e}_k$, only the k-th row of the full embedding matrix $\boldsymbol{E}$ was used. The gradient $\frac{\partial L}{\partial \boldsymbol{E}}$ is therefore a sparse matrix, zero everywhere except for the k-th row. This can be constructed via an outer product with the one-hot vector $\boldsymbol{e}_k$:

$$\frac{\partial L}{\partial \boldsymbol{E}} = \boldsymbol{e}_k \left(\frac{\partial L}{\partial \boldsymbol{v}_k}\right)^\intercal = \boldsymbol{e}_k \left(\boldsymbol{W}_1^\intercal \boldsymbol{\delta}_2\right)^\intercal. \tag{B20}$$

During training, this ensures that only the embedding for the specific token present in the input is updated. Over many examples, this process sculpts meaningful representations into the embedding vectors.

Appendix C: The Self-Attention Mechanism

Let us consider a sequence of n input tokens—say, an "unfinished" sentence—to be finished by the token derived from the output Y in Eq. (C7). Each token is represented by a learned embedding vector, and these row vectors are stacked to form an input matrix $X \in \mathbb{R}^{n\times d}$, where d is the dimension of the embedding space (often called d_{model} in literature). The self-attention mechanism computes an output matrix $Y \in \mathbb{R}^{n\times d}$ where each output row $\boldsymbol{y}_i$ is a new, contextualized representation of the corresponding input token $\boldsymbol{x}_i$. This is achieved through the following steps.

1. Projection to Query, Key, and Value

First, the input matrix X is linearly projected into three distinct matrices: Query ($\boldsymbol{Q}$), Key ($\boldsymbol{K}$), and Value ($\boldsymbol{V}$). This is accomplished using three learned weight matrices: $\boldsymbol{W}_Q \in \mathbb{R}^{d\times d_k}$, $\boldsymbol{W}_K \in \mathbb{R}^{d\times d_k}$, and $\boldsymbol{W}_V \in \mathbb{R}^{d\times d_v}$. Typically, for the single-head attention block, the dimensions are set such that $d = d_k = d_v$ [2] (for multi-head attention with h parallel heads discussed in D 1, $d_k = d_v = d/h$).

$$\boldsymbol{Q} = X\boldsymbol{W}_Q \in \mathbb{R}^{n\times d_k}, \tag{C1}$$

$$\boldsymbol{K} = X\boldsymbol{W}_K \in \mathbb{R}^{n\times d_k}, \tag{C2}$$

$$\boldsymbol{V} = X\boldsymbol{W}_V \in \mathbb{R}^{n\times d_v}. \tag{C3}$$

Intuitively, for each token, these projections serve different purposes:

1. The **Query** vector can be seen as a representation of what information the current token is seeking from the rest of the sequence.
2. The **Key** vector represents what kind of information the token itself contains or offers.
3. The **Value** vector is the actual content or representation of the token that will be passed on if other tokens attend to it.

2. Attention Score Calculation

The relationship, or compatibility, between each pair of tokens is computed by taking the dot product of their respective Query and Key vectors. This is performed for all tokens simultaneously via a matrix multiplication between $\boldsymbol{Q}$ and the transpose of $\boldsymbol{K}$. The resulting matrix of raw attention scores is:

$$\text{Scores} = \boldsymbol{Q}\boldsymbol{K}^T \in \mathbb{R}^{n\times n}. \tag{C4}$$

The element at position (i, j) in this matrix represents the raw relevance of token j to token i. This matrix is then scaled by the square root of the key dimension, d_k, to ensure numerical stability during training. The scaling is crucial because for large values of d_k, the dot products can grow very large, pushing the subsequent softmax function into regions where its gradient is near zero, which would stall the learning process. The scaled scores are therefore:

$$\text{Scaled Scores} = \frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}. \tag{C5}$$

3. Applying Softmax and Causal Masking

A softmax function is applied row-wise to the scaled scores matrix. This converts the raw, unnormalized scores into a probability distribution, yielding the attention weights matrix A.

$$A = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{n\times n}. \tag{C6}$$

Each element A_{ij} is the weight assigned to the Value vector of token j when computing the output for token i, and the sum of each row $\sum_j A_{ij}$ is 1.

For autoregressive models that generate text one token at a time (like GPT), a *causal mask* is applied before the softmax step. This prevents a token at position i from attending to any subsequent tokens (i.e., tokens at positions $j > i$), which would be equivalent to "seeing into the future." The mask is implemented by adding $-\infty$

to all elements in the upper-triangular part of the scaled scores matrix (where $j > i$). When the softmax function is applied, these $-\infty$ values become zero, effectively nullifying the connections to future tokens.

4. Output Computation

The final output of the self-attention layer is a weighted sum of all Value vectors, where the weights are given by the matrix A. This computation is performed with a single matrix multiplication:

$$Y = A\boldsymbol{V} \in \mathbb{R}^{n \times d_v}. \tag{C7}$$

The i-th row of the output, $\boldsymbol{y}_i$, is therefore a contextualized vector for the i-th token, formed by aggregating information from all other tokens in the sequence to which it was allowed to attend.

The entire self-attention operation can be expressed compactly as:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{C8}$$

A loss function is calculated to quantify the divergence between the predicted token distribution and the ground-truth token—the target that is the actual token completion—from the training set. The gradient of this loss is then backpropagated through the model to adjust all learnable parameters, including the embedding matrix and the query (W_Q), key ($\boldsymbol{W}_K$), and value ($\boldsymbol{W}_V$) matrices. (For multi-head attention below, this includes the output matrix $\boldsymbol{W}_O$.)

Appendix D: Advanced Mechanisms

While the scaled dot-product attention is the core of the Transformer, its true power comes from how this mechanism is integrated into a robust and repeatable block structure. Two key advancements enable this: Multi-Head Attention and the specific architecture of the Transformer block itself.

1. Multi-Head Attention

A single attention mechanism might be forced to learn an "average" of several types of relationships between tokens. To allow the model to capture a richer set of relationships, the Transformer employs *multi-head attention*. This mechanism does not run a single attention calculation, but rather multiple attention calculations in parallel.

The intuition is to allow the model to jointly attend to information from different representational subspaces. For instance, one head might learn to track syntactic dependencies, while another focuses on semantic similarity. This is achieved as follows:

1. **Projection:** Instead of one set of projection matrices, the model learns h independent sets of matrices ($\boldsymbol{W}_{Q,i}, \boldsymbol{W}_{K,i}, \boldsymbol{W}_{V,i}$ for each head $i = 1, \ldots, h$), with $d_k = d_v = d/h$. The input X is projected through each of these sets, creating h different Query, Key, and Value representations.

2. **Parallel Attention Calculation:** The standard attention function is applied to each of these h projected sets in parallel, yielding h distinct output matrices, or "heads."

$$\text{head}_i = \text{Attention}(X\boldsymbol{W}_{Q,i}, X\boldsymbol{W}_{K,i}, X\boldsymbol{W}_{V,i}). \tag{D1}$$

3. **Concatenation and Final Projection:** The outputs of these h heads are "horizontally" concatenated along the feature dimension into a single, larger matrix. This matrix is then projected back to the original model dimension d using an additional learned weight matrix, $\boldsymbol{W}_O \in \mathbb{R}^{hd_v \times d}$, which learns to aggregate the information from all heads.

$$\text{MultiHead}(X) = \text{Concat}[\text{head}_1, \ldots, \text{head}_h]\boldsymbol{W}_O. \tag{D2}$$

2. The Complete Transformer Block

A complete Transformer "block" is the fundamental repeating unit of the model, designed to facilitate the stable training of very deep networks. Each block is composed of two main sub-layers: the multi-head attention mechanism followed by a position-wise feed-forward network. Crucially, each of these sub-layers is wrapped with two additional components: a residual connection and layer normalization.

Let the input to a block be Z. The data flows through the block as follows:

1. **Multi-Head Attention Sub-layer:** First, the multi-head attention is computed on the input Z.

2. **Residual Connection and Layer Normalization ("Add & Norm"):** The output of the attention mechanism is added to the original input Z (a residual connection), and the result is then normalized using Layer Normalization.

$$Z' = \text{LayerNorm}(Z + \text{MultiHead}(Z)). \tag{D3}$$

3. **Feed-Forward Network (FFN) Sub-layer:** The intermediate output Z' is then passed through a two-layer, position-wise feed-forward network. This network processes each token's representation independently and identically.

4. **Second "Add & Norm":** A final residual connection and layer normalization are applied.

$$\text{Output} = \text{LayerNorm}(Z' + \text{FFN}(Z')). \tag{D4}$$

The roles of these components are critical:

1. **Position-wise FFN:** This network introduces additional computational capacity, allowing the model to learn more complex transformations of the token representations after information has been aggregated by the attention mechanism.

2. **Residual Connections:** These "shortcuts" allow the gradient to flow more directly through the network during backpropagation. It helps combat the vanishing gradient problem and makes it easier for the model to learn an identity mapping for a given block, effectively allowing it to dynamically adjust its own depth during training.

3. **Layer Normalization:** This technique stabilizes the network by normalizing the activations of each token's representation. By keeping the mean and variance of the activations consistent, it ensures a smoother training process and reduces dependence on careful parameter initialization.

These blocks are then stacked multiple times—often dozens of times—to form the full LLM. Each block refines and enriches the token representations, allowing the model to build up increasingly complex and abstract understandings of the input sequence.

3. An Embedding-Centric View of the Transformer

The mathematical operations of the Transformer, particularly self-attention, are best understood as a sophisticated mechanism for the iterative refinement of token representations. The entire architecture is designed to transform an initial set of static, context-free embeddings into a final set of dynamic, context-rich embeddings that encode deep relational information from the input sequence.

a. Initial State: Context-Free Embeddings

The process begins when an input text sequence is converted into a matrix of initial token embeddings, $X \in \mathbb{R}^{n \times d}$. At this stage, each row vector $\boldsymbol{x}_i$ is context-free. For example, the initial embedding for the token "bank" is identical in the phrases "river bank" and "money bank." This initial vector represents a generalized, pre-trained meaning of the word, akin to a dictionary average, but it lacks any awareness of its specific role in the given context.

b. The Role of the Transformer Block

The primary function of a Transformer block is to take a set of token embeddings as input and produce a new, more refined set of embeddings as output. The self-attention mechanism drives this transformation by enabling a form of "conversation" among the token embeddings. In this process:

1. Each token's embedding (via its **Query** projection) effectively queries all other token embeddings in the sequence.

2. It assesses their relevance based on a compatibility function with their respective **Key** projections.

3. It then synthesizes its new representation by computing a weighted sum of their **Value** projections, where the weights are determined by the aforementioned relevance scores.

The output of the block is therefore a new matrix of embeddings where each token's vector representation has absorbed relevant contextual information from its peers. After just one layer, the embedding for "bank" in "river bank" will have integrated information from the "river" token, making its vector representation distinct from that of "bank" in "money bank."

c. Hierarchical Refinement through Stacking

The power of the Transformer architecture comes from stacking these blocks sequentially. The output embeddings of block k become the input embeddings for block $k + 1$. This creates a processing hierarchy where representations become progressively more abstract and sophisticated:

1. **Lower Layers:** The first few blocks typically resolve local ambiguities and encode basic syntactic information. They perform the initial step of contextualization, disambiguating word senses as in the "bank" example.

2. **Intermediate Layers:** These blocks operate on already-contextualized embeddings. They can therefore model more complex and longer-range relationships, such as syntactic dependencies, coreference resolution (e.g., linking a pronoun to its antecedent), and semantic roles.

3. **Upper Layers:** The final blocks work with highly abstract representations, allowing them to capture thematic, pragmatic, and discourse-level information from the entire sequence.

Thus, the statement that "each block refines and enriches the token representations" is the central concept. The Transformer is a machine designed to evolve a set of vectors from simple dictionary lookups into nuanced, context-aware carriers of meaning. The emphasis is unequivocally on the token embeddings as the dynamic computational substrate of the model.